



LLM Self Evaluation

LLM Self-Evaluation

1) What it is & why it matters

LLM self-evaluation prompts a model to **assess its own output** for quality, accuracy, policy compliance, or format adherence—then **repair** if needed.

Benefits:

- **Higher reliability** (catch errors before users/systems see them)
 - **Better adherence** (format, length, tone, schema)
 - **Faster iteration** (feedback for prompt optimization)
 - **Lower ops cost** (fewer human corrections)
-

2) Core patterns (choose the simplest that works)

1. Critique → Revise

- Produce answer **A**, generate a **critique**, then return **A'** that fixes issues.

2. Rubric Scoring

- Score against a **task-specific rubric** (1–5 or pass/fail) with short evidence.

3. Checklist/Contract Adherence

- Validate required sections/keys/limits; return missing or invalid items.

4. Grounding/Factuality Check

- Cross-check claims against provided context/citations; flag unsupported statements.

5. Uncertainty/Confidence Estimation

- Return a calibrated confidence **0–1** with reasons and decision thresholds.

6. Safety/Policy Gate

- Evaluate for disallowed content; emit a safe fallback if violated.

Tip: Keep self-eval short and structured. You want crisp signals, not more prose.

3) When to use (and when not)

Use when

- Outputs must follow **schemas** or strict formats (JSON, forms)
- **Reasoning** or **extraction** accuracy is critical
- You need **guardrails** (policy, safety, PII)
- You're **A/B testing prompts** and need quick feedback signals

Avoid / limit when

- Ultra-low latency or tiny budgets (skip extra passes)
 - The same model can be easily **biased** by seeing its own answer (prefer a **separate judge prompt** or smaller judge model)
-

4) Design principles

- **Separation of roles**: different instructions (or a different model) for **Author** vs **Judge** reduces bias.
 - **Constrained outputs**: ask for **JSON** with booleans, scores, and *short* notes.
 - **Fail-fast**: if checks fail, request a **targeted repair** (don't re-write everything).
 - **Determinism**: low temperature for judging ($T \approx 0-0.3$).
 - **Evidence > opinion**: require quotes, spans, or citation IDs from the given context.
-

5) Ready-to-use templates (copy-paste)

5.1 Critique → Revise (generic)

Task: Produce output A to the user request below.
Then self-evaluate A against the checklist and return a corrected A'.

Return ONLY the final corrected output, preceded by:

FINAL:

<your corrected answer>

Checklist:

- Follows required format and length
- Answers all parts of the question
- Uses only provided context (no inventions)
- Style: <tone/audience>

User request:

<<<PROMPT>>>

Context (authoritative):

<<<CONTEXT>>>

5.2 Rubric Judge (scalar score + notes)

Evaluate the candidate answer against this rubric. Be strict and concise.

Rubric (score 1–5):

- 1) Task fidelity (followed instructions exactly)
- 2) Factual grounding (uses only <CONTEXT>)
- 3) Completeness (addresses all parts)
- 4) Format adherence (schema/sections/length)

Return ONLY JSON:

```
{  
  "scores": {"fidelity": n, "grounding": n, "completeness":  
n, "format": n},  
  "overall": n,
```

```
"notes": "≤40 words, key issues only"
}
```

CONTEXT:

<<<CONTEXT>>>

CANDIDATE:

<<<ANSWER>>>

5.3 Contract Adherence (schema check)

Check if ANSWER matches the required contract.

Contract:

- Sections: #Overview, #Pros, #Cons, #Action Items
- Word limits: Overview ≤80; each list ≤4 bullets
- No external links or emojis

Return ONLY JSON:

```
{
  "valid": true|false,
  "violations": ["missing:#Pros", "overview>80w", "has_links"],
  "repair_plan": "≤30 words describing minimal edits"
}
```

ANSWER:

<<<ANSWER>>>

5.4 Grounding/Factuality with citations

Using only CONTEXT, verify the CANDIDATE.

For each claim, mark supported/unsupported and cite the context span ID if supported.

Return ONLY JSON:

```
{
```

```
"unsupported_claims": [...],
"supported_claims": [{"text": "...", "sourceId": "DOC3#p
2"}],
"overall_grounded": true|false
}
```

CONTEXT:

<<<DOCS WITH IDS>>>

CANDIDATE:

<<<ANSWER>>>

5.5 Confidence / Uncertainty

Given the task and output, estimate confidence 0–1.
Calibrate: 0.9 means ~9/10 similar cases should be correct.

Return ONLY JSON:

```
{ "confidence": 0.0-1.0, "reasons": ["short bullets"], "needs_human_review": true|false }
```

5.6 Safety/Policy Gate

Evaluate CANDIDATE for policy violations (PII, hate, self-harm, medical/financial advice).

If violated, return ONLY:

```
{"violation": true, "category": "<label>", "replacement": "<safe fallback text>"}
```

Else return:

```
{"violation": false}
```

6) Minimal refinement loop (controller)

1. **Author pass** → produce answer A
2. **Judge pass** → run rubric + contract checks on A
3. **If pass** → return A

4. **If fail** → prompt **Repair**: provide `violations` and `repair_plan`; get `A'`
5. **Re-check** → stop after 1–2 repair rounds (guard latency)

Pseudocode

```
A = author(prompt, context)
J = judge(rubric, contract, A, context)
if J.valid && J.overall >= THRESH: return A
A' = repair(A, J.violations, J.repair_plan, context)
J' = judge(..., A', context)
return (J'.valid ? A' : fallback_message)
```

7) Metrics & calibration

- **Adherence**: schema validity %, required-sections %, length violations %
- **Quality**: rubric overall mean, pass rate at threshold
- **Factuality**: unsupported claims per 100 answers
- **Safety**: violation rate, false positives/negatives
- **Confidence calibration**: Brier score / ECE on a labeled set
- **Cost/latency**: extra tokens from judge/repair passes

Calibrate confidence by comparing predicted confidence vs. actual correctness on a dev/hold-out set; adjust thresholds for auto-approve vs. human review.

8) Pitfalls & mitigations

Pitfall	Mitigation
Self-leniency / bias	Separate Author and Judge prompts (or models); low temperature for Judge
Verbose judging	Enforce JSON with short <code>notes</code> ; limit tokens
False “grounding”	Require source IDs/quotes ; reject claims without citations
Endless repair loops	Cap to 1–2 rounds; return safe fallback
Cost spikes	Judge only high-risk tasks; sample; cache validated outputs

Pitfall	Mitigation
Over-strict rules	Start with minimal must-haves; expand gradually

9) Quick checklist (copy-paste)

- ☐ Separate **Author** and **Judge** roles (or models)
- ☐ Judge uses **structured JSON** outputs, low temperature
- ☐ Rubric + contract tailored to the task; short notes
- ☐ Grounding requires **source IDs**; unsupported claims flagged
- ☐ One or two **repair** attempts max, then fallback
- ☐ Track **adherence, factuality, safety, latency, cost**
- ☐ Calibrate **confidence** thresholds; route low-confidence to review
- ☐ Version prompts; keep a regression suite of past failures

Key Takeaway

Self-evaluation turns an LLM from a one-shot responder into a **check-and-repair pipeline**. Keep roles separated, signals structured, and loops short—so you gain reliability without sacrificing speed.